

Actividad. Análisis de expresión diferencial funcional

Javier Luque Serrano

2026-02-06

Table of Contents

Preprocesado de los datos	2
Cargar las librerías.....	2
Obtención del dataset.....	3
Ideal.....	3
Nuestro caso	4
Exploración del dataset.....	5
Arreglar data frame <i>brca_exp_mat</i>	7
Arreglar data frame <i>clinical_info</i>	8
Análisis de expresión diferencial	13
Normalización	14
Exploración de los datos normalizados	17
Expresión diferencial de los genes	19
Análisis de enriquecimiento funcional	25
Similitud semántica (Gene Ontology)	25
Análisis de Enriquecimiento del set de Genes (GSEA).....	28
Base de datos KEGG.....	31
Análisis de sobrerrepresentación (ORA)	32
Base de datos KEGG.....	35
Conclusiones	39
Bibliografía	41
Declaración de uso de IA	41

En esta actividad se llevará a cabo un análisis de expresión diferencial funcional sobre el conjunto de datos de cáncer de mama BRCA, recogido por el proyecto TCGA (The Cancer Genome Atlas). Es decir, vamos a tratar de entender la diferencia en la expresión de determinados genes en una muestra de tejido sano respecto a tejido

tumoral, de manera que se puedan clasificar bien los genes que afectan más a la proliferación del cáncer.

Preprocesado de los datos

Cargar las librerías

Antes de empezar, se van a cargar las librerías que se necesitan para hacer el análisis.

```
# Normalización de la matriz de expresion (incluido en limma)
library(edgeR)

## Loading required package: limma

# Analisis de expresion diferencial
library(limma)
# Descargar Los datos de TCGA
#library(TCGAbiolinks)
#library(SummarizedExperiment)
# Hacer gráficas
library(ggplot2)
# Complementario a ggplot2: graficas mas detalladas
library(ggrepel)
# Heatmap
library(pheatmap)
# Similitud semántica
library(GOSemSim)

## GOSemSim v2.34.0 Learn more at https://yulab-smu.top/contribution-
knowledge-mining/
##
## Please cite:
##
## Guangchuang Yu. Gene Ontology Semantic Similarity Analysis Using
## GOSemSim. In: Kidder B. (eds) Stem Cell Transcriptional Networks.
## Methods in Molecular Biology. 2020, 2117:207-215. Humana, New York,
NY.

# Base de datos
library(org.Hs.eg.db)

## Welcome to Bioconductor
##
##      Vignettes contain introductory material; view with
##      'browseVignettes()'. To cite Bioconductor, see
##      'citation("Biobase)", and for packages 'citation("pkgname)".

# Libreria Disease Ontology
library(DOSE)
```

```
## DOSE v4.2.0 Learn more at https://yulab-smu.top/contribution-
knowledge-mining/
##
## Please cite:
##
## Guangchuan Yu, Li-Gen Wang, Guang-Rong Yan, Qing-Yu He. DOSE: an
## R/Bioconductor package for Disease Ontology Semantic and Enrichment
## analysis. Bioinformatics. 2015, 31(4):608-609

# Enriquecimiento funcional
library(clusterProfiler)
## Please cite:
##
## T Wu, E Hu, S Xu, M Chen, P Guo, Z Dai, T Feng, L Zhou, W Tang, L
Zhan,
## X Fu, S Liu, X Bo, and G Yu. clusterProfiler 4.0: A universal
## enrichment tool for interpreting omics data. The Innovation. 2021,
## 2(3):100141

# Manejo de dataframes
library(dplyr)

# Vías metabólicas en KEGG
library(AnnotationDbi)
# Compilar RMarkdown como PDF
library(knitr)
library(tinytex)
```

Obtención del dataset

Ideal

En esta sección vamos a acceder a la base de datos GDC (Genomics Data Commons) para descargar los datos TCGA-BRCA.

```
# Ver La Lista de proyectos que hay en GDC
#gdcprojects <- getGDCprojects() # Lista de proyectos disponibles
#getProjectSummary('TCGA-BRCA') # Resumen del proyecto TCGA-BRCA

# Construimos una query con la información que queremos del proyecto
#query_TCGA <- GDCquery(
#  project = "TCGA-BRCA",
#  data.category = "Transcriptome Profiling",
#  data.type = "Gene Expression Quantification",
#  workflow.type = "STAR - Counts" # Datos crudos de la expresión de los
genes en cada muestra
#)

# Ver un resumen del objeto query_TCGA, el cual es una lista
#resultados <- getResults(query_TCGA)
```

```
# Descargamos Los datos
#GDCdownload(query_TCGA)

# Preparar Los datos en un objeto SummarizedExperiment
#BRCA_data <- GDCprepare(query_TCGA, summarizedExperiment = T)
```

De esta manera, hemos obtenido un objeto del tipo SummarizedExperiment, el cual es una clase de Bioconductor, diseñado para almacenar datos genómicos complejos, que contiene:

- Expresión de los genes en cada muestra. Típicamente, las filas son los genes y las columnas, las muestras.
- Información sobre los genes en las filas, como el nombre, IDs, ubicación en el cromosoma, etc.
- Información sobre las muestras en las columnas, como el ID del paciente e información clínica como la edad, género, estadio de tumor, etc.

Se puede acceder a cada parte de la siguiente manera:

```
# Acceder a La matriz de expresión (conteos)
#counts_matrix <- assay(BRCA_data)

# Acceder a La información de Los genes como un dataframe
#gene_info <- as.data.frame(rowData(BRCA_data))

# Acceder a La información clínica y de Las muestras como un dataframe
#clinical_info <- as.data.frame(colData(BRCA_data))
```

Debido a la gran cantidad de datos a descargar, se va a proceder utilizando los datasets proporcionados para la actividad, previamente divididos en matriz de expresión y datos clínicos.

Nuestro caso

```
library(readr)

clinical_info <- read_tsv("clinical_info_TCGA-BRCA.tsv", show_col_types = FALSE,
                        na=c("NA", "", "[Not Applicable]", "[Not Available]", "[Unknown]",
                            "[Not Evaluated]", "Indeterminate", "[Discrepancy]"))

brca_exp_mat <- read_tsv("BRCA_exp_matrix.tsv", show_col_types = FALSE,
                        na=c("NA", "", "[Not Applicable]", "[Not Available]", "[Unknown]",
```

```

                                "[Not
Evaluated]", "Indeterminate", "[Discrepancy]"))

# Convertirlos en data frames, que son mas manejables que el formato
tibble
brca_exp_mat <- as.data.frame(brca_exp_mat)
clinical_info <- as.data.frame(clinical_info)

```

Al cargar los datos aparece un aviso que tendremos que arreglar más adelante.

Exploración del dataset

Si trabajamos con los datos descargados, habría que utilizar el dataframe *gene_info* para construir la matriz de conteos únicamente con los genes que son codificantes, eliminar duplicados y utilizar el nombre del gen en lugar de su identificador *ensemble_id*, de manera que la visualización sea más representativa.

Como vamos a usar los datasets que se ofrecen en la actividad, nos saltaremos el paso de crear la matriz de expresión a partir de los metadatos de los genes. Aún así, es imprescindible hacer un análisis exploratorio para comprobar que los datos (tanto la matriz de expresión como la matriz de datos clínicos) están listos para trabajar con ellos, es decir, que no haya duplicados, que tengan el mismo número de muestras, etc.

```

## Exploracion del dataset
dim(brca_exp_mat)

## [1] 20500 1155

dim(clinical_info)

## [1] 1177 114

# Comprobar primeras filas y columnas
head(brca_exp_mat[, 1:5]) # Primera columna es una muestra, hay que
desplazar el dataset

##   SP3930 SP3918 SP4007 SP4019 SP3995
## 1  ALPL    278    181   2731    167
## 2  ALPK1   2189   1498   2798   1347
## 3  ALPK2    140    284    232    143
## 4  ALPK3    607   1072    420    353
## 5 ALPPL2     0     0      1     0
## 6  ALPP     0     0     70     0

#una columna a la derecha
head(brca_exp_mat[, 1150:1155])

##   SP12885 SP12870 SP12856 SP12974 SP12956      SP12938
## 1   2457    678    910    419   2696 182\t      69
## 2   2730   1042    617   2351    784 699\t     975

```

```
## 3      534      62      385      68      25 101\t      61
## 4      326     396     345     387     314 363\t     244
## 5        0        0        0        0        0  0\t        0
## 6        1        1        0        0        0  0\t        0

head(clinical_info[, 1:5]) # Primera columna es una muestra, hay que
desplazar el dataset

##      snames      sample bcr_patient_uuid
bcr_patient_barcode
## 1      833 TCGA-E2-A15C      SP3930 da840a22-8d30-4682-bbd0-
8fa87bd40528
## 2      446 TCGA-AR-A250      SP3918 a0963e93-b63e-4de2-86c1-
81302e271d3e
## 3       92 TCGA-A2-A0YM      SP4007 5cbf0aea-ebb4-4005-bdd5-
14ef2dc6826c
## 4      423 TCGA-AR-A1AS      SP4019 7d681cc6-689d-41c8-9e84-
e13733089ec9
## 5      320 TCGA-AN-A0FL      SP3995 baf1207d-7d1a-4cad-ba58-
62c26be2415e
## 6     1008 TCGA-GM-A2DM      SP3983 753D5D49-3B40-4737-9D4C-
B1EC29CC2692
##      form_completion_date
## 1      TCGA-E2-A15C
## 2      TCGA-AR-A250
## 3      TCGA-A2-A0YM
## 4      TCGA-AR-A1AS
## 5      TCGA-AN-A0FL
## 6      TCGA-GM-A2DM

#una columna a la derecha

# Comprobar duplicados
table(duplicated(colnames(brca_exp_mat)))

##
## FALSE
## 1155

table(duplicated(brca_exp_mat$gene_ID))

## < table of extent 0 >

# Comprobar valores NA en los nombres de las filas
sum(is.na(brca_exp_mat[, 1]))

## [1] 0
```

Aquí se observa un problema importante de resolver: la primera columna, la cual representa el nombre de los genes, tiene el nombre de una muestra, mientras que la última columna tiene los datos de expresión de dos muestras, lo cual tenemos que

arreglar para no perder información. Ocurre lo mismo en *clinical_info*, donde todos los nombres de las columnas están desplazados hacia la izquierda y no corresponden con sus valores.

Arreglar data frame *brca_exp_mat*

```
# Cargamos Librerias
library(tidyr) # para modificar la ultima columna

##
## Attaching package: 'tidyr'

## The following object is masked from 'package:S4Vectors':
##
##      expand

library(stringr) # para eliminar el espacio que hay alrededor del \t

# Guardar Las columnas que tenemos
current_colnames <- colnames(brca_exp_mat)
# Definir La ultima columna para separar Los valores
last_col_name <- current_colnames[length(current_colnames)]
# Limpiar el espacio en blanco alrededor del tabulador
brca_exp_mat[[last_col_name]] <- str_trim(brca_exp_mat[[last_col_name]])
brca_exp_mat[[last_col_name]] <-
  str_replace_all(brca_exp_mat[[last_col_name]],
                  "\\s*\\t\\s*", "\\t")

# Separar La ultima columna en dos con nombres temporales
brca_exp_mat <- brca_exp_mat %>%
  separate_wider_delim(
    cols = all_of(last_col_name),
    delim = "\\t",
    names = c("second_to_last", "very_last"),
    too_few = "align_start"
  )

# Desplazar Las columnas hacia La derecha y nombrar a La columna 1
"gene_ID"
old_names <- current_colnames[1:(length(current_colnames) - 2)]
new_colnames <- c("gene_ID", old_names,
                 current_colnames[length(current_colnames) - 1],
                 current_colnames[length(current_colnames)])

colnames(brca_exp_mat) <- new_colnames

brca_exp_mat <- as.data.frame(brca_exp_mat)

# Establecer La columna de genes como el nombre de Las filas
rownames(brca_exp_mat) <- brca_exp_mat[, 1]
brca_exp_mat <- brca_exp_mat[, -1]
```

```
# Verificar
dim(brca_exp_mat)

## [1] 20500 1155

head(colnames(brca_exp_mat))

## [1] "SP3930" "SP3918" "SP4007" "SP4019" "SP3995" "SP3983"

tail(colnames(brca_exp_mat))

## [1] "SP12885" "SP12870" "SP12856" "SP12974" "SP12956" "SP12938"
```

Arreglar data frame *clinical_info*

En este caso, se van a desplazar los nombres de las columnas hacia la derecha y se va a eliminar la primera columna, la cual contiene valores numéricos que no son representativos de las muestras. Sin embargo, al eliminar la última columna, estamos perdiendo la información de *tumor_tissue_site*, la cual podría ser importante. Aún así, ante la imposibilidad de descargar los datos, es la única estrategia que veo posible.

```
### Desplazar Los nombres de Las columnas hacia La derecha
# Coger Las n-1 columnas
col_names_correct <-
colnames(clinical_info)[1:(length(colnames(clinical_info))-1)]
# Crear nombres nuevos: un marcador para la primera columna y el resto
desplazados
temporal_names <- c("delete", col_names_correct)
# Asignar Los nuevos nombres a Las columnas
colnames(clinical_info) <- temporal_names
# Eliminar La primera columna
clinical_info <- clinical_info[, -1]

# Verificar
head(clinical_info[, 1:5])

##          snames sample          bcr_patient_uuid
bcr_patient_barcode
## 1 TCGA-E2-A15C SP3930 da840a22-8d30-4682-bbd0-8fa87bd40528
TCGA-E2-A15C
## 2 TCGA-AR-A250 SP3918 a0963e93-b63e-4de2-86c1-81302e271d3e
TCGA-AR-A250
## 3 TCGA-A2-A0YM SP4007 5cbf0aea-ebb4-4005-bdd5-14ef2dc6826c
TCGA-A2-A0YM
## 4 TCGA-AR-A1AS SP4019 7d681cc6-689d-41c8-9e84-e13733089ec9
TCGA-AR-A1AS
## 5 TCGA-AN-A0FL SP3995 baf1207d-7d1a-4cad-ba58-62c26be2415e
TCGA-AN-A0FL
## 6 TCGA-GM-A2DM SP3983 753D5D49-3B40-4737-9D4C-B1EC29CC2692
```

TCGA-GM-A2DM

```
##   form_completion_date
## 1      2011-4-13
## 2      2011-8-30
## 3      2010-12-8
## 4      2011-6-14
## 5      2010-12-28
## 6      2011-9-30
```

```
head(clinical_info[, 110: 113])
```

```
##   project_code site_of_primary_tumor_other stage_other
tissue_source_site
## 1      <NA>          NA          NA
E2\tBreast
## 2      <NA>          NA          NA
AR\tBreast
## 3      <NA>          NA          NA
A2\tBreast
## 4      <NA>          NA          NA
AR\tBreast
## 5      <NA>          NA          NA
AN\tBreast
## 6      <NA>          NA          NA
GM\tBreast
```

Finalmente tenemos la matriz de expresión con todas las filas y columnas correctamente dispuestas, donde la primera columna es la columna de nombres de los genes. Proseguimos con el análisis exploratorio.

Comprobar si hay genes duplicados en la informacion clinica

```
table(duplicated(clinical_info$sample))
```

```
##
## FALSE  TRUE
## 1172    5
```

Eliminamos los duplicados con el operador inverso ! y el vector logico de duplicados

```
clinical_info_filtered <-
clinical_info[!duplicated(clinical_info$sample),]
```

Comprobar que todas las muestras de clinical_info estén en brca_exp_mat y viceversa

```
table(clinical_info_filtered$sample%in%colnames(brca_exp_mat))
```

```
##
## FALSE  TRUE
##    17 1155
```

```
table(colnames(brca_exp_mat)%in%clinical_info_filtered$sample)
```

```
##
## TRUE
## 1155

# Ver las 17 muestras que sobran en clinical_info respecto a brca_exp_mat
clinical_patients <- clinical_info_filtered$sample
clinical_patients[!clinical_patients %in% colnames(brca_exp_mat)]

## [1] "SP3768" "SP3774" "SP3261" "SP3266" "SP5690" "SP8418"
##      "SP3016"
## [8] "SP5100" "SP5106" "SP4484" "SP4490" "SP9684" "SP12063"
##      "SP12070"
## [15] "SP8502" "SP10814" "SP10821"

# Eliminar las muestras sobrantes
clinical_info_filtered <- clinical_info[clinical_info$sample %in%
colnames(brca_exp_mat), ]

# Comprobar que tienen la misma dimension y estan en el mismo orden
all(colnames(brca_exp_mat)==clinical_info_filtered$sample)

## [1] TRUE

# Comprobar primeras filas y columnas
head(brca_exp_mat[, 1:5])

##          SP3930 SP3918 SP4007 SP4019 SP3995
## ALPL         278    181   2731    167   1540
## ALPK1        2189   1498   2798   1347    424
## ALPK2         140    284    232    143     15
## ALPK3         607   1072    420    353    155
## ALPPL2         0      0      1      0      0
## ALPP          0      0     70      0      0

# Comprobamos las dimensiones de las matrices y sus clases
dim(clinical_info_filtered) # Filas: muestras; columnas: variables
clinicas

## [1] 1155  113

class(clinical_info_filtered)

## [1] "data.frame"

dim(brca_exp_mat)

## [1] 20500  1155

class(brca_exp_mat)

## [1] "data.frame"
```

```
# Ver un resumen de ambos datasets
```

```
summary(brca_exp_mat[, 1:5])
```

```
##      SP3930      SP3918      SP4007      SP4019
## Min.   :    0   Min.   :    0   Min.   :    0   Min.   :    0
## 1st Qu.:   13   1st Qu.:   15   1st Qu.:   16   1st Qu.:    5
## Median :  494   Median :   709   Median :   641   Median :   357
## Mean   : 2214   Mean   :  3448   Mean   :  3109   Mean   :  2040
## 3rd Qu.: 2097   3rd Qu.:  3058   3rd Qu.:  2854   3rd Qu.:  1746
## Max.   :419665   Max.   :1079565   Max.   :332492   Max.   :366558
##      SP3995
## Min.   :    0
## 1st Qu.:    4
## Median :   288
## Mean   :  2174
## 3rd Qu.:  1653
## Max.   :1249045
```

```
summary(clinical_info_filtered[, 1:5])
```

```
##      snames      sample      bcr_patient_uuid
bcr_patient_barcode
## Length:1155      Length:1155      Length:1155      Length:1155
## Class :character Class :character Class :character Class
:character
## Mode  :character Mode  :character Mode  :character Mode
:character
## form_completion_date
## Length:1155
## Class :character
## Mode  :character
```

Finalmente, tenemos la matriz de expresión génica y la información clínica de las pacientes filtrada, con unas dimensiones coherentes entre sí.

Vemos que en la información clínica había 5 duplicados y 17 muestras más que no aparecían en la matriz de expresión génica, por lo que se han eliminado. Así, nos quedamos con dos data frames que contienen información de **1155 muestras**: por un lado, la **expresión de 20500 genes** y por otro, **114 columnas con la información de las variables clínicas e información de las muestras**.

Sin embargo, para evitar errores futuros en la regresión lineal, vamos a comprobar cómo son los datos en la columna *er_status_by_ihc*, ya que los valores NA nos darán problema, ya que harán que la matriz de diseño no tenga la misma dimensión que la matriz de expresión. Además, vamos a corroborar que ningún paciente que tenga algún valor en esa columna, tenga también la variable *age_at_diagnosis* para poder hacer bien la regresión.

```
# Ver que tipo de muestras hay
```

```
table(clinical_info_filtered$er_status_by_ihc)
```

```
##
## Negative Positive
##      224      759

# Ver cuantos valores NA hay
sum(is.na(clinical_info_filtered$er_status_by_ihc))

## [1] 172

sum(is.na(clinical_info_filtered$age_at_diagnosis))

## [1] 123
```

Como podemos ver, hay 983 muestras con las que vamos a poder trabajar, ya que hay 172 muestras de las que se desconoce el estatus del receptor de estrógenos. Por lo tanto, se van a eliminar esas muestras (filas) en el data frame de *clinical_info* y sus correspondientes (columnas) en la matriz de expresión, *brca_exp_mat*. De igual manera, se eliminarán las muestras de las que se desconozca la edad de diagnóstico.

```
# Eliminar muestras que tengan tumor_status == NA
clinical_info_clean <-
clinical_info_filtered[!is.na(clinical_info_filtered$er_status_by_ihc), ]

# Ver si quedan valores NA
sum(is.na(clinical_info_clean$er_status_by_ihc))

## [1] 0

sum(is.na(clinical_info_clean$age_at_diagnosis))

## [1] 0
```

Ahora, debemos eliminar las muestras correspondientes en la matriz de expresión.

```
# Obtener el ID de las muestras en ambos datasets
samples_exp <- colnames(brca_exp_mat)
samples_clinic <- clinical_info_clean$sample

# Buscar muestras presentes en ambos datasets
common_samples <- intersect(samples_exp, samples_clinic)

cat("Muestras de la matriz de expresión:", length(samples_exp), "\n")
## Muestras de la matriz de expresión: 1155

cat("Muestras de los datos clínicos:", length(samples_clinic), "\n")
## Muestras de los datos clínicos: 983

cat("Muestras comunes:", length(common_samples), "\n")
```

```
## Muestras comunes: 983

# Crear subset de Las muestras comunes
brca_exp_clean <- brca_exp_mat[, common_samples]

# Verificar que Las dimensiones cuadran
ncol(brca_exp_clean)

## [1] 983

nrow(clinical_info_clean)

## [1] 983

# Verificar que el orden es el mismo
all(colnames(brca_exp_clean) == clinical_info_clean$sample)

## [1] TRUE
```

Así, el número de muestras se ha reducido a **983**. Finalmente, comprobamos la distribución de las variables que nos interesan:

```
table(clinical_info_clean$er_status_by_ihc)

##
## Negative Positive
##      224      759

table(clinical_info_clean$age_at_diagnosis)

##
## 26 27 28 29 30 31 32 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49
## 50 51 52
##   2   1   1   5   1   3   1   9   5 10   6 11 13 21 16 11 14 14 23 26 25 26 23
## 29 28 26
## 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75
## 76 77 78
## 21 30 20 28 17 26 29 20 34 46 26 23 22 23 19 21 20 17 19 12 17 14 12
## 12 12 13
## 79 80 81 82 83 84 85 86 87 88 89 90
## 14 15   7   6   5 10   5   1   1   4   3   9
```

Es probable que si hubiéramos usado la función `read.delim()` no tuviéramos que haber hecho todo esto, pero es bueno saberlo.

Análisis de expresión diferencial

Ya una vez se ha comprobado que las matrices con las que vamos a trabajar están bien ordenadas, debemos entrar en en análisis de expresión diferencial, donde se va a comparar la expresión de los genes de ambos grupos de muestras: muestras con receptor de estrógenos negativo, *ER-* (casos) vs. muestras con receptor de

estrógenos positivo, *ER+* (control). Esta, **er_status_by_ihc** será nuestra **variable independiente**, ya que es la que estamos analizando.

Por otro lado, tenemos la edad de diagnóstico, **age_at_diagnosis**, que será nuestra covariable, ya que es la variable de ajuste (o control) que incluimos para eliminar posibles variaciones de la edad sobre los resultados; no queremos ver qué genes cambian de expresión con la edad, queremos tener en cuenta la edad para ver cómo se expresan los genes en el estatus de ER.

Normalización

Este paso es imprescindible para corregir los valores de la cuantificación con el fin de reducir los sesgos técnicos y permitir determinar de forma más precisa los valores de expresión y diferencias biológicas relevantes.

Aquí vamos a usar el paquete *edgeR* para normalizar, el cual está dentro del paquete *limma*. Este paquete utiliza un tipo específico de objeto llamado *DGEList*, el cual tenemos que crear utilizando la matriz de conteos que queremos normalizar.

Tenemos que tener en cuenta que, dentro de los 20500 genes, habrán algunos que no se expresen, o que estén poco expresados, lo cual puede dar lugar a falsos positivos. Así, se va a llevar a cabo una normalización del tipo **Conteos por Millón (CPM)**, la cual consiste en normalizar los valores de conteo por el número total de lecturas secuenciadas, calculadas en millones de lecturas.

Para elegir el umbral de la expresión mínima de un gen, vamos a tener en cuenta el tamaño del grupo más pequeño, observando la variable *er_status_by_ihc*, que ya vimos anteriormente que la clase *NEGATIVE* tenía **224 muestras**. Por lo tanto, estableceremos que un gen tiene que estar expresado en, al menos, 224 muestras para ser tenido en cuenta. Además, cada expresión representará 1 millón de lecturas (*cpm* = 1).

```
### Normalizacion: Limpiar conteo quitando genes poco expresados
# Convertimos el data frame en una matriz
exp_matrix <- as.matrix(brca_exp_clean)
class(exp_matrix) <- "numeric" # Forzar a que la clase sea numerica para
poder crear la
#matriz de expresion

# Creamos una matriz de expresión
de.exp <- DGEList(counts=exp_matrix) # Este objeto espera una matriz.

# Nos quedamos con genes que estén expresados en, al menos, 19 muestras
keep.exprs <- rowSums(cpm(de.exp)>1)>=224 # Vector booleano
# Vemos cuantos genes estan poco expresados
table(keep.exprs)
```

```
## keep.exprs
## FALSE TRUE
## 5533 14967

# Creamos un subset de las filas (genes) que tienen expresion en mas de
19 muestras
de.exp <- de.exp[keep.exprs,] # Objeto DGEList
# Vemos cuantos genes quedan despues de la filtracion
dim(de.exp)

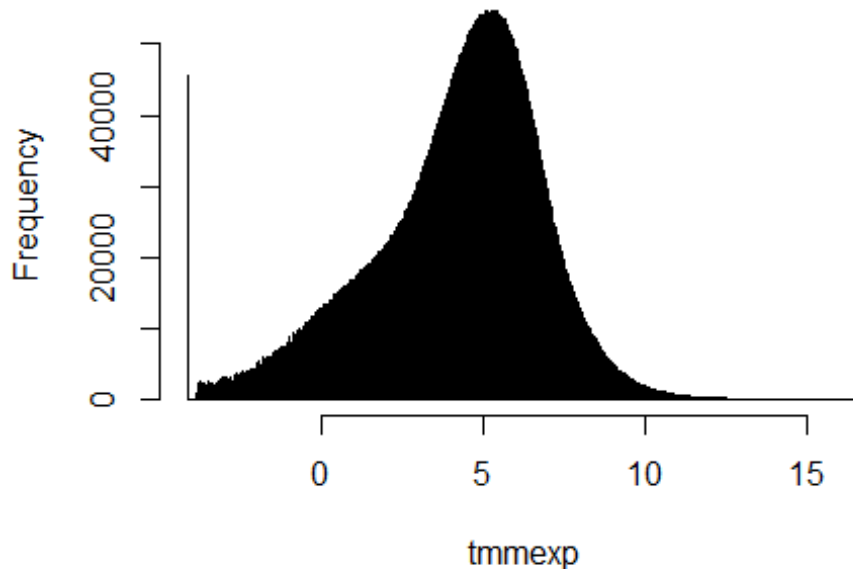
## [1] 14967 983
```

Al calcular la dimensión de *de.exp* vemos que 5533 genes no están expresados en, como mínimo, 224 muestras, por lo que han sido eliminados. Así, hemos normalizado los valores de conteo por el número total de lecturas secuenciadas.

Entonces, utilizando únicamente los genes bien expresados, vamos a normalizar los valores de cuantificación en función del tamaño de la librería, para lo cual usaremos el algoritmo TMM (*Trimmed Mean of M-Values*).

```
# Normalizacion TMM
limma <- calcNormFactors(de.exp, method = "TMM")
# Extraemos la matriz de conteos normalizada
tmexp <- cpm(limma, log = T, prior.count = 3) #
counts*1000000/(libs.size*fact)
# Hacemos un histograma de la expresion de los genes normalizados
hist(tmexp, 1000, main = paste("Histograma de la matriz de conteos
normalizada "))
```

Histograma de la matriz de conteos normalizada



Este histograma, representados con una transformación logarítmica para que se visualicen bien todos los genes, ya que, como se observa, hay una gran cantidad de genes con poca expresión, mientras que los genes que están más representados tienen una frecuencia menor.

A continuación, vamos a comparar los datos no normalizados frente a los normalizados mediante un diagrama de cajas, de manera que se pueda ver la diferencia en la variabilidad de la expresión de los genes. Para ello, se va a crear un subset de 15 muestras.

```
# Graficar siempre las mismas 15 muestras
set.seed(8)
samples_indices = sample(1:ncol(de.exp), 15)

# Crear matriz de conteos sin normalizar
no_tmmexp <- cpm(de.exp, log = T, prior.count = 3)

# Crear subsets de 15 muestras
no_tmmexp_subset <- no_tmmexp[, samples_indices]
tmmexp_subset <- tmmexp[, samples_indices]

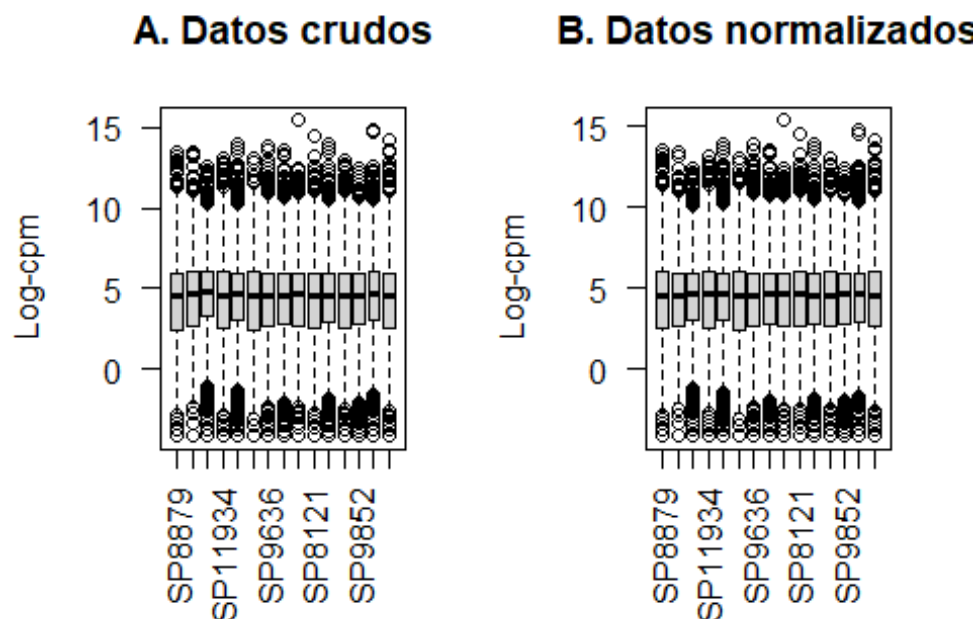
# Boxplot de datos no normalizados vs. normalizados
par(mfrow=c(1,2), oma = c(0, 0, 2, 0)) # oma añade margen externo
boxplot(no_tmmexp_subset, las=2, main="")
title(main="A. Datos crudos", ylab="Log-cpm")
boxplot(tmmexp_subset, las=2, main="")
```

```

title(main="B. Datos normalizados",ylab="Log-cpm")
mtext("Efecto de la normalización TMM sobre 15 muestras aleatorias",
      side = 3, line = 0, outer = TRUE, cex = 1.3, font = 2) # Añade
titulo principal

```

de la normalización TMM sobre 15 muestras ale



```

par(mfrow=c(1,1))

```

Se puede observar que hay mucha más variabilidad en la figura A, siendo, por lo tanto, menos interpretable. A su vez, el diagrama de cajas de la figura B con los datos normalizados, tiene una mediana mucho más homogénea, de manera que es mucho más potente estadísticamente.

Exploración de los datos normalizados

A continuación, una práctica imprescindible es hacer un PCA (*Principal Component Analysis*) sobre los datos normalizados, lo que nos muestra la variabilidad entre las muestras usando sólo los datos de expresión en dos dimensiones. Así podemos detectar rápidamente si las muestras se dividen de manera clara entre muestras de ER- y muestras de ER+.

Este análisis nos da una primera vista clara de cómo se van a comportar las muestras y de cuales deben ser nuestros próximos pasos.

Es importante destacar que este PCA espera el nombre de los genes en las columnas, por lo que si queremos ver la expresión diferencial de las muestras, debemos hacer la traspuesta. Además, antes de hacer la gráfica, es importante

descomponer la variabilidad de los datos en el porcentaje de importancia de cada uno de los componentes que explican dicha variabilidad en los datos. Esto se hace con la desviación estándar de los distintos componentes.

Finalmente, haremos una gráfica con los dos componentes principales, es decir, los que más aportan a la variabilidad de las muestras.

```
# PCA despues de normalizacion
pca_results <- prcomp(t(tmmexp), scale. = TRUE)

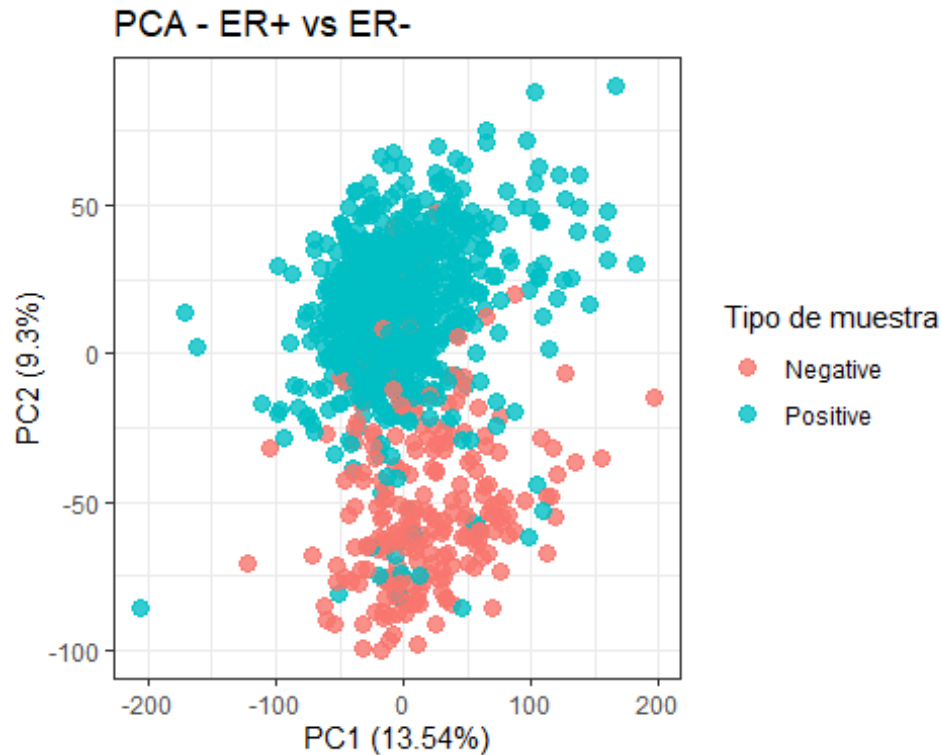
# Calcular varianza explicada
var_explained <- pca_results$sdev^2 / sum(pca_results$sdev^2) * 100

# Preparar datos para ggplot
pca_data <- data.frame(
  PC1 = pca_results$x[,1],
  PC2 = pca_results$x[,2],
  sample = rownames(pca_results$x)
)

# Añadir metadatos clínicos
pca_data <- merge(pca_data, clinical_info_filtered[,c("sample",
"tumor_status", "gender",
                                                    "age_at_diagnosis",
"bcr_patient_barcode", "er_status_by_ihc")],
  by.x = "sample", by.y = "sample")

# Gráfico PCA
pca_plot <- ggplot(pca_data, aes(x = PC1, y = PC2, color =
er_status_by_ihc)) +
  geom_point(size = 3, alpha = 0.8) +
  labs(title = "PCA - ER+ vs ER-",
    x = paste0("PC1 (", round(var_explained[1], 2), "%)" ),
    y = paste0("PC2 (", round(var_explained[2], 2), "%)" ),
    color = "Tipo de muestra") +
  theme_bw() +
  theme(legend.position = "right")

# Mostrar grafico
print(pca_plot)
```



Podemos ver que hay cierta diferenciación entre genes que dan positivo en receptor de estrógenos y aquellos que dan negativo. Esto es buena señal.

Expresión diferencial de los genes

Vamos a continuar, con el análisis con *limma* para ver la expresión diferencial de los genes.

Para empezar, tenemos que crear una matriz con la información de las muestras en binario (0 para muestras tumorales, 1 para muestras normales). Esta será la matriz de diseño (también conocida como matriz de contrastes).

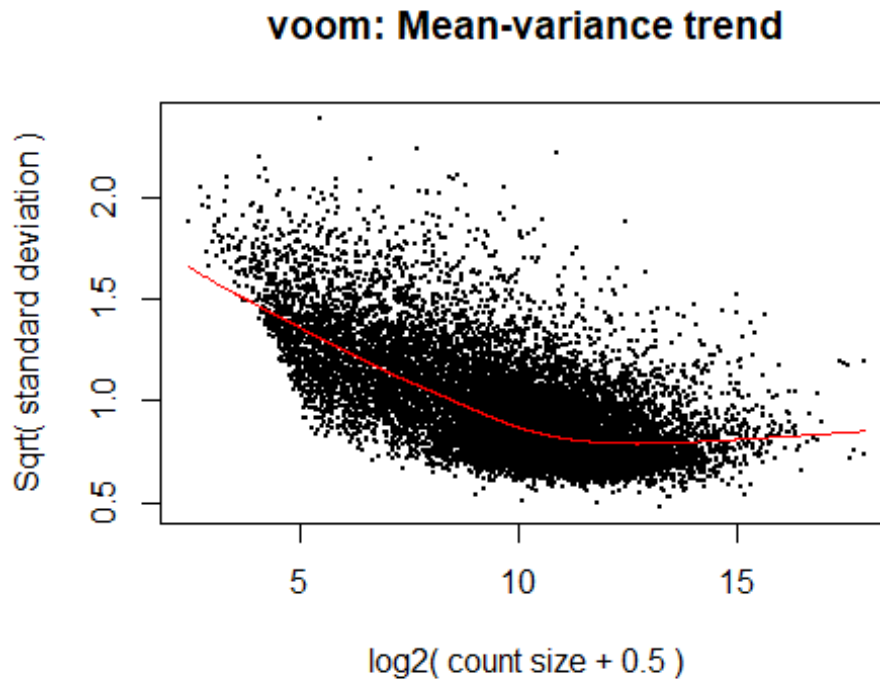
```
# Crear matriz de diseño
design <- model.matrix(~0+clinical_info_clean$er_status_by_ihc +
                      clinical_info_clean$age_at_diagnosis)

# Limpiar los nombres de las columnas
colnames(design) <- gsub("clinical_info_clean$er_status_by_ihc", "",
                        colnames(design), fixed = T)
colnames(design) <- gsub("clinical_info_clean$age_at_diagnosis", "age",
                        colnames(design), fixed = T)
```

Haciendo esto, ya tenemos los dos objetos que necesita *limma* para hacer la transformación de los datos con *voom* y la **regresión lineal**: el objeto DGEList *limma* y la matriz de diseño *design*.

Como limma fue creado para el análisis de microarrays, la función voom espera datos continuos para la transformación. Sin embargo, estamos trabajando con la expresión de los genes mediante conteos, por lo que son variables discretas. Por ello, le vamos a pedir que represente en el diagrama los valores de expresión de cada gen en función de su varianza.

```
# Hacemos la gráfica de voom
fit <- voom(limma, design, plot = T)
```



```
# Regresión lineal con la expresión de los datos y su varianza
fit <- lmFit(fit, design)
```

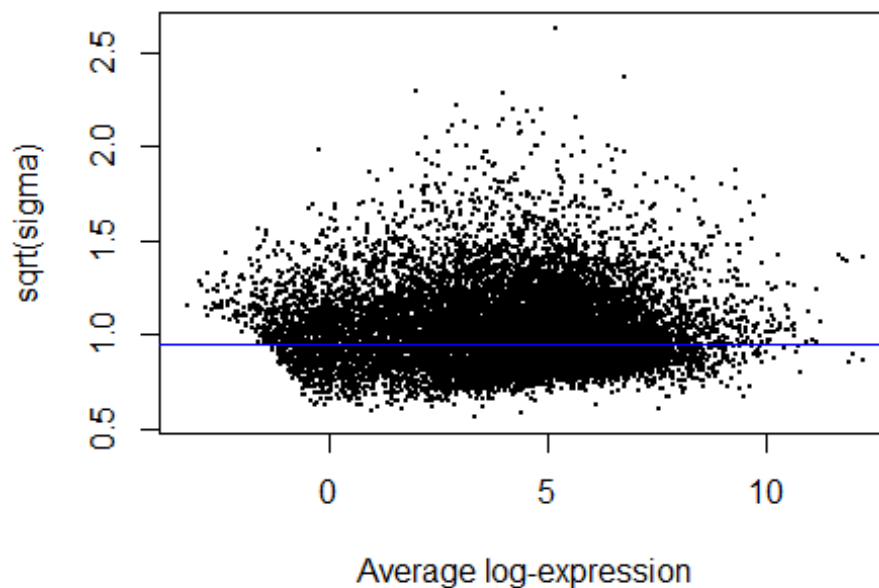
Se puede observar que los genes que tienen una menor expresión (más hacia la izquierda) tienen una mayor varianza (más hacia arriba), lo cual es lo esperable. Además, la transformación ha funcionado bien, pues la línea roja se adapta bien a la distribución, por lo que podemos proceder con la regresión lineal.

A continuación, haremos un contraste donde se compararán todas las muestras que tengan tejido tumoral frente a las que tienen tejido normal (en este sentido, para que los valores de Fold Change queden en positivo).

```
# Contrastes
# Comparacion de tumores vs tejido normal
cont.wt <- makeContrasts(er_p_vs_n=Negative - Positive, levels=design)
# Hacemos la regresion lineal con los contrastes
fit2 <- contrasts.fit(fit, cont.wt)
```

Finalmente, hacemos una inferencia bayesiana para obtener información más homogénea a lo largo de todas las muestras, infiriendo la varianza de cada uno de los genes. Funciona muy bien cuando tenemos pocas muestras.

```
# Aproximacion bayesiana
fit2 <- eBayes(fit2)
plotSA(fit2)
```



Obtenemos un gráfico con los valores de las varianzas transformados, de manera que tenemos una varianza homogénea a lo largo de toda la distribución, independientemente de la expresión de los genes.

Una vez completado este análisis, debemos obtener una tabla con los genes que están diferencialmente expresados obtenida de la función Bayes.

```
# Obtener tabla de genes diferencialmente expresados
de_res<-topTable(fit2,coef = "er_p_vs_n",number = Inf) #inf para que
devuelva todos Los genes
table(de_res$adj.P.Val<0.05)

##
## FALSE TRUE
## 2980 11987
```

Se debe tener en cuenta el p-valor ajustado, ya que se resuelve la significancia aleatoria que pueda obtener un gen debido a gran cantidad de muestras sobre las que se calcula el p-valor. Así, obtenemos que **11987 genes están diferencialmente**

expresados. Este número puede parecer muy alto (representa el 58% de la cantidad total de genes), sin embargo, puede tener sentido ya que el estatus de receptor de estrógenos deriva a subtipos de cancer diferentes, debido a distintas rutas metabólicas, distintos programas celulares o, incluso, distintas redes regulatorias de genes.

Aún así, podemos filtrar más, añadiendo el logaritmo del Fold Change, *logFC*, que representa la proporción de aumento que se produce en la expresión; se suele poner en base 2. En este caso, representará que las muestras de receptor de estrógenos negativa, *ER-* están doblemente expresadas sobre las de *ER+*.

```
table(de_res$adj.P.Val<0.05 & abs(de_res$logFC)>1)
```

```
##  
## FALSE TRUE  
## 13035 1932
```

Vemos que sólo 1932 genes cumplen ambas condiciones, por lo que 13035 genes no serán significantes. Vamos a hacer una gráfica de volcán, o *volcanoplot*, donde se verá la significancia de la expresión de los genes diferenciados, es decir, los genes *upregulados* y *downregulados*.

```
#### Volcano plot  
# Si se cumplen ambas condiciones, será significativo. Si además se cumple  
# que logFC > 0,  
# será upregulated.  
de_res$Significance <- ifelse(de_res$adj.P.Val < 0.5 & abs(de_res$logFC)  
> 1,  
                             ifelse(de_res$logFC > 0, "Upregulated",  
                                     "Downregulated"),  
                             "Not Significant")  
  
# Guardamos el nombre de los genes en una nueva columna  
de_res$ID<-rownames(de_res)  
  
# Ordenar los datos por significancia (adj.P.Val) para seleccionar el top  
10  
top_genes <- de_res[order(de_res$adj.P.Val), ][1:10, ]  
  
# Definimos los parámetros del gráfico  
volcano_plot <- ggplot(de_res, aes(x = logFC, y = -log10(adj.P.Val),  
color = Significance)) +  
  geom_point(alpha = 0.8, size = 2) + # Puntos del gráfico  
  scale_color_manual(values = c("Upregulated" = "red", "Downregulated" =  
"blue",  
                                "Not Significant" = "grey")) +  
  theme_minimal() + # Tema limpio  
  labs(title = "Volcano Plot",  
        x = "Log2 Fold Change",
```

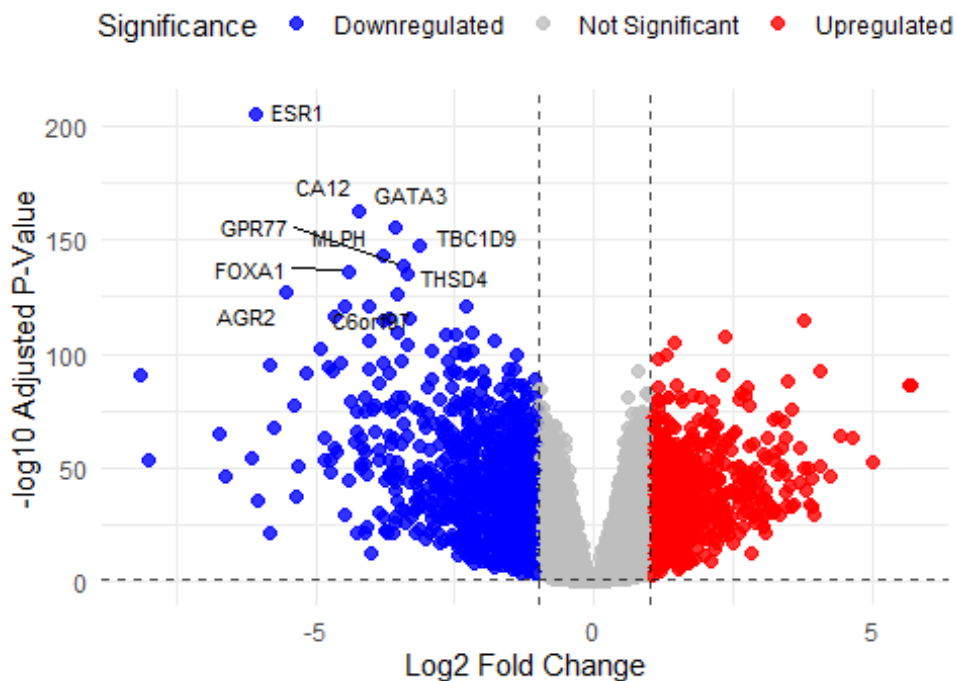
```

y = "-log10 Adjusted P-Value") +
# Líneas de FC
geom_vline(xintercept = c(-1, 1), linetype = "dashed", color = "black",
alpha = 0.7) +
# Línea de p-valor
geom_hline(yintercept = -log10(0.05), linetype = "dashed", color =
"black", alpha = 0.7) +
theme(legend.position = "top") +
geom_text_repel(data = top_genes, aes(label = ID),
size = 3, col="black",
box.padding = 0.3,
point.padding = 0.2,
max.overlaps = Inf)

#Pintamos el plot
volcano_plot

```

Volcano Plot



Aquí vemos el Fold Change en el valor de la x y el p-valor ajustado en el eje y, con los genes upregulados (a la izquierda en azul), lo que significa que dichos genes están más expresados significativamente en muestras con ER+ que en muestras con ER-, mientras los genes downregulados (a la derecha en rojo), están más expresados significativamente en todas las muestras con ER+.

Más adelante, en el análisis funcional, se verán más en detalle la expresión de algunos genes y la función que tienen.

Finalmente en este primer análisis, vamos a hacer un heatmap, que es básicamente una matriz de conteo con los datos normalizados para los 10 genes más importantes de ambos tipos, upregulados y downregulados.

```
### heatmap
# Seleccionar los 10 genes más upregulados y los 10 genes más
downregulados
top_upregulated <- de_res[de_res$Significance == "Upregulated", ]
top_upregulated <- top_upregulated[order(-top_upregulated$logFC), ][1:10,
]

top_downregulated <- de_res[de_res$Significance == "Downregulated", ]
top_downregulated <- top_downregulated[order(top_downregulated$logFC),
][1:10, ]

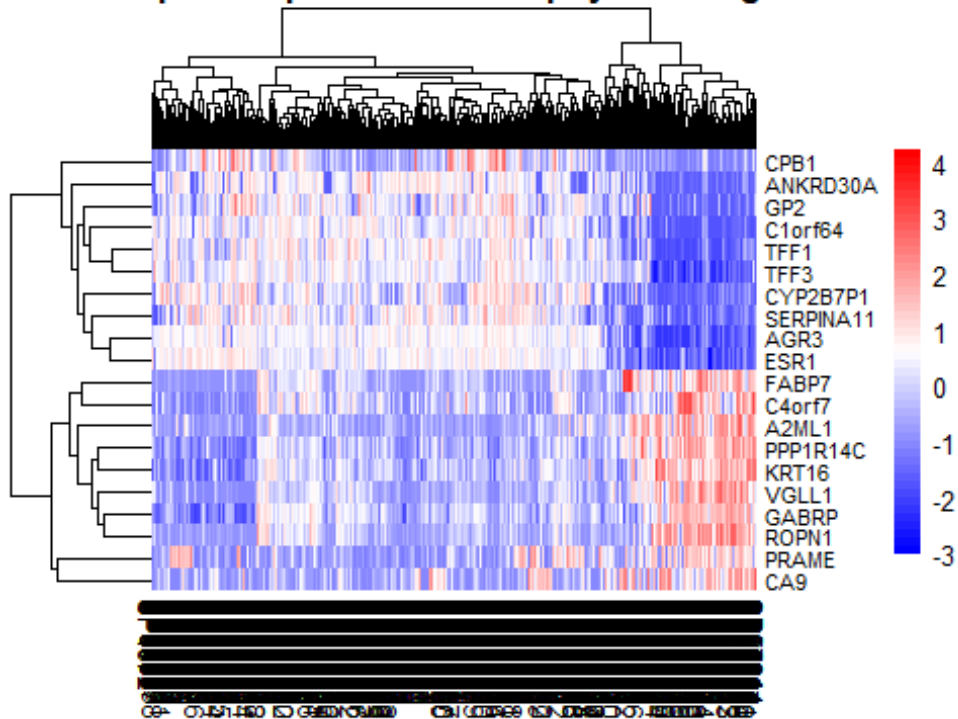
# Combinar los 20 genes seleccionados
top_genes <- rbind(top_upregulated, top_downregulated)

# Extraer la matriz de expresión para estos genes
heatmap_data <- tmmexp[rownames(tmmexp) %in% top_genes$ID, ]

# Escalar los datos por filas (genes) para mejor visualización
scaled_data <- t(scale(t(heatmap_data))) # Escalado por fila

# Crear el heatmap
pheatmap(scaled_data,
  color = colorRampPalette(c("blue", "white", "red"))(50), #
  Escala de colores
  cluster_rows = TRUE, # Agrupar genes
  cluster_cols = TRUE, # Agrupar muestras
  show_rownames = TRUE, # Mostrar nombres de genes
  show_colnames = TRUE, # Mostrar nombres de muestras
  fontsize_row = 8, # Tamaño de la fuente para los nombres de los
genes
  fontsize_col = 10, # Tamaño de la fuente para los nombres de
las muestras
  main = "Heatmap del Top 20 de Genes Up- y Downregulados") #
Título del heatmap
```

Heatmap del Top 20 de Genes Up- y Downregulados



Se puede ver que hay una gran acumulación de genes upregulados para muestras que tienen ER+ en la parte alta a la derecha (en azul), mientras que las muestras que están más downreguladas (en rojo) se encuentran clusterizadas también.

En la tabla *top_genes* encontramos que el **gen que está mejor diferencialmente expresado es el ESR1**, lo cual es una muy buena señal, ya que este es el gen de receptor de estrógenos 1, el cual es el gen que codifica la proteína de receptores de estrógenos, lo cual tiene una alta implicación biológica.

Análisis de enriquecimiento funcional

Similitud semántica (Gene Ontology)

En este análisis, deberemos inferir la función de los genes, anotando de manera similar los genes que tienen una función similar, lo que se conoce como **similitud semántica**. Es decir, la información disponible en unos genes nos sirve para inferir la información faltante de otros.

Así, haremos una **matriz de similitud** de los 20 genes asociados que hemos obtenido previamente.

En primer lugar, utilizaremos la base de datos **Gene Ontology (GO)**, la cual recoge la función de las proteínas en los organismos con un vocabulario preciso y dinámico, aunque sin asociación a una enfermedad.

Para hacer esto, hay varias aproximaciones. Aquí vamos a usar el método de **DAGs** (*Directed Acyclic Graph*), el cual utiliza grafos dirigidos que no tiene ciclos para calcular la similitud entre dos términos mediante su localización en el grafo y las relaciones con los términos ancestros. Así, se utilizará el método *Wang*, y como estamos comparando múltiples términos semánticos, lo combinaremos con *Best-Match Average (BMA)*, el cual computa la media de todos los valores de similitud máximos para cada fila y columna.

```
# Llamamos al archivo de la base de datos GO, donde Hs es Homo Sapiens
# utilizando la ontología molecular function
hsGO <- godata('org.Hs.eg.db', keytype = "SYMBOL", ont = "MF",
computeIC=FALSE)
```

```
## Warning in godata("org.Hs.eg.db", keytype = "SYMBOL", ont = "MF",
computeIC =
## FALSE): use 'annoDb' instead of 'OrgDb'
```

```
## preparing gene to GO mapping data...
```

```
# Creamos la lista de genes con la que trabajar
genes <- rownames(top_genes)
# Calcular la matriz de similitud con el metodo Wang combinado con BMA
mgeneSim(genes, semData = hsGO, measure = "Wang", combine = "BMA",
verbose=FALSE)
```

```
##          VGLL1 A2ML1 FABP7 ROPN1 KRT16    CA9 PPP1R14C GABRP PRAME
AGR3  TFF1
## VGLL1    1.000 0.218 0.603 0.737 0.596 0.428    0.162 0.095 0.553
0.737 1.000
## A2ML1    0.218 1.000 0.129 0.129 0.129 0.129    0.359 0.064 0.129
0.129 0.129
## FABP7    0.603 0.129 1.000 0.728 0.616 0.466    0.100 0.098 0.577
0.728 1.000
## ROPN1    0.737 0.129 0.728 1.000 0.753 0.533    0.100 0.096 0.679
0.894 1.000
## KRT16    0.596 0.129 0.616 0.753 1.000 0.439    0.100 0.103 0.566
0.753 1.000
## CA9      0.428 0.129 0.466 0.533 0.439 1.000    0.100 0.070 0.431
0.533 1.000
## PPP1R14C 0.162 0.359 0.100 0.100 0.100 0.100    1.000 0.042 0.100
0.100 0.100
## GABRP    0.095 0.064 0.098 0.096 0.103 0.070    0.042 1.000 0.087
0.096 0.128
## PRAME    0.553 0.129 0.577 0.679 0.566 0.431    0.100 0.087 1.000
0.679 1.000
## AGR3     0.737 0.129 0.728 0.894 0.753 0.533    0.100 0.096 0.679
1.000 1.000
## TFF1     1.000 0.129 1.000 1.000 1.000 1.000    0.100 0.128 1.000
1.000 1.000
## GP2      0.676 0.129 0.679 0.833 0.692 0.492    0.100 0.103 0.630
```

```

0.833 1.000
## ESR1      0.499 0.434 0.497 0.508 0.472 0.447      0.320 0.106 0.483
0.492 1.000
## CPB1      0.474 0.129 0.511 0.600 0.487 0.377      0.100 0.080 0.470
0.600 1.000
## TFF3      0.737 0.129 0.728 1.000 0.753 0.533      0.100 0.096 0.679
0.894 1.000
##          GP2  ESR1  CPB1  TFF3
## VGLL1     0.676 0.499 0.474 0.737
## A2ML1     0.129 0.434 0.129 0.129
## FABP7     0.679 0.497 0.511 0.728
## ROPN1     0.833 0.508 0.600 1.000
## KRT16     0.692 0.472 0.487 0.753
## CA9       0.492 0.447 0.377 0.533
## PPP1R14C  0.100 0.320 0.100 0.100
## GABRP     0.103 0.106 0.080 0.096
## PRAME     0.630 0.483 0.470 0.679
## AGR3      0.833 0.492 0.600 0.894
## TFF1      1.000 1.000 1.000 1.000
## GP2       1.000 0.484 0.551 0.833
## ESR1      0.484 1.000 0.456 0.508
## CPB1      0.551 0.456 1.000 0.600
## TFF3      0.833 0.508 0.600 1.000

```

La dimensión máxima de esta matriz de similitud aquí en la interfaz es de 15, por lo que no podemos ver la similitud semántica de los 20 genes. Vemos que la mayoría de genes no están muy relacionados. Sin embargo, podemos ver un caso interesante como es el del gen TFF1 o TFF2, ambos reguladores clásicos de ER, los cuales tienen una correlación semántica de 1 con muchos otros genes, lo cual puede estar indicando algún problema en la calidad de los datos o en el procesamiento. Por otro lado, el gen estrella del receptor de estrógenos, ESR1, tiene una correlación media con la mayoría de genes ER-regulados, lo cual puede ser esperado, porque aunque regula otros genes, no están co-expresados al mismo nivel.

Vamos a comparar directamente el grupo de genes upregulados con el grupo de genes downregulados, para ver qué correlación tienen.

```

# Medir la correlación de Los 10 genes upregulados con Los 10 genes
downregulados
gs1 <- rownames(top_upregulated)
gs2 <- rownames(top_downregulated)
GOSemSim::clusterSim(
  gs1, gs2, semData = hsGO, measure = "Wang", combine = "BMA")
## [1] 0.574

```

Ambos grupos tienen una correlación media, lo cual tiene sentido, pues unos activan a otros, mientras que otros inhiben a algunos otros. Debemos ir más allá para entender estas correlaciones.

Por lo tanto, utilizando la base de datos Gene Ontology, se procederá a hacer el análisis de enriquecimiento funcional.

Análisis de Enriquecimiento del set de Genes (GSEA)

Este primer análisis de enriquecimiento funcional nos permitirá introducir la información resultante del análisis de expresión diferencial anterior. Es robusto estadísticamente, ya que utiliza todos los genes significativos que hemos obtenido en el anterior análisis, considerando la magnitud de cambios de expresión ($\log FC$).

Se divide en 3 pasos clave: - Cálculo de un score de enriquecimiento (ES), mediante el parámetro de Kolmogorov-Sirnov (KS) enriquecido. - Estimación del nivel de significación del parámetro de puntuación calculado, mediante un p-valor test de permutaciones. - Finalmente, se hace una corrección por múltiples comparaciones (FDR) para evitar resultados significativos por azar.

Es muy importante tener en cuenta varias cosas. Para empezar, las funciones esperan un vector ordenado de manera descendente por el Fold Change, es decir, los genes más arriba serán los que están más upregulados y los que estén más abajo en la lista, serán los que están más downregulados. Por lo tanto, tenemos que modificar nuestro set de genes, que ahora mismo es un data frame. Además, la anotación GO en *OrgDb* está indexada por ENTREZID. Como aquí tenemos una lista de genes que hemos obtenido nosotros, tenemos que adaptarlo.

Por lo tanto, teniendo en cuenta que nuestro **universo de genes son los 14967 genes diferencialmente expresados**, contenidos en el objeto *de_res*, y nuestra **lista de genes serán los 1932 genes significativos**, objeto que vamos a crear ahora.

```
# Guardamos la lista de genes significativos
signif_genes_df <- de_res[de_res$Significance == "Downregulated" |
de_res$Significance == "Upregulated", ]
# Lo convertimos en vector
signif_genes <- sign(signif_genes_df$logFC) * (-
log10(signif_genes_df$P.Value))
# Le damos nombre a los genes
names(signif_genes) <- rownames(signif_genes_df)
# Ordenar en orden descendente
signif_genes <- sort(signif_genes, decreasing = TRUE)
# Comprobar el objeto
str(signif_genes)

## Named num [1:1932] 118 110 108 102 101 ...
## - attr(*, "names")= chr [1:1932] "PSAT1" "B3GNT5" "TTLL4" "IFRD1" ...
```

A continuación, se va a calcular sobre este vector de genes bien catalogado el enriquecimiento, para lo cual se va a utilizar la ontología de **procesos biológicos (BP)**, pues es la que más interés biológico tiene, ya que los tumores ER+ y ER- difieren en procesos como factores de crecimiento en la activación de vías metabólicas,

redes neuronales de respuesta a estrógenos o en la proliferación de firmas hormonales; todos procesos biológicos, no actividades moleculares ni localizaciones celulares.

Aquí no tenemos que tener en cuenta el universo de genes.

```
# Analisis de enriquecimiento de genes
ego <- gseGO(geneList = signif_genes,
             OrgDb = org.Hs.eg.db,
             ont = "BP", # Biological Process
             keyType = "SYMBOL",
             minGSSize = 50, # minimo 50 genes asociados para el termino
             maxGSSize = 300, # maximo 300 genes asociados para el
termino
             pvalueCutoff = 0.05,
             eps = 0,
             verbose = FALSE)
# min: valor minimo de genes con los que trabajo. idem max
head(ego, 3)

##              ID              Description setSize
enrichmentScore
## G0:0022402 G0:0022402          cell cycle process      121
0.4758466
## G0:0000280 G0:0000280          nuclear division        59
0.6060979
## G0:1903047 G0:1903047 mitotic cell cycle process      82
0.5395375
##              NES          pvalue      p.adjust          qvalue rank
## G0:0022402 2.884051 1.535732e-14 6.081501e-12 5.156828e-12 543
## G0:0000280 3.179673 5.353118e-14 1.059917e-11 8.987604e-12 537
## G0:1903047 3.063734 1.849973e-13 2.441964e-11 2.070672e-11 543
##              leading_edge
## G0:0022402 tags=69%, list=28%, signal=53%
## G0:0000280 tags=80%, list=28%, signal=59%
## G0:1903047 tags=74%, list=28%, signal=56%
##
core_enrichment
## G0:0022402
CCNE1/CENPW/LPIN1/E2F3/CDC20/PHGDH/CENPA/CENPN/CDCA2/CDCA8/CHEK1/TTK/CDC2
5A/AURKB/STMN1/SKP2/KIF2C/ROPN1B/LYN/EGFR/FOXM1/FANCA/RAD54L/NDC80/HJURP/
MELK/CCNB2/EZH2/CEP55/PLK1/SMC1B/PLCG2/STIL/CDK6/ANLN/CCNA2/KIFC1/NFIB/GP
R3/BUB1/CDC45/TGFA/RAD51AP1/SKA3/MYBL2/SKA1/TRIP13/TPX2/CDCA5/BLM/SFRP1/C
DC25B/DSCC1/DIAPH3/BIRC5/FAM83D/DLGAP5/CDKN2A/NCAPH/UBE2C/CDT1/PTTG1/KIF1
8B/MND1/NCAPG/SLC16A1/KIF14/KIF20A/NUF2/KIF18A/KIF4A/AURKA/MKI67/ASPM/EIF
4EBP1/GATA6/KIF15/ID4/RRM2/CENPF/STAG3/CENPE/IQGAP3
## G0:0000280
CCNE1/CDC20/CDCA2/CDCA8/CHEK1/TTK/AURKB/KIF2C/FANCA/RAD54L/NDC80/CCNB2/PL
K1/ANLN/KIFC1/NFIB/GPR3/BUB1/TGFA/RAD51AP1/SKA3/MYBL2/SKA1/TRIP13/TPX2/CD
CA5/CDC25B/BIRC5/DLGAP5/NCAPH/UBE2C/CDT1/PTTG1/KIF18B/MND1/NCAPG/KIF14/NU
```

```

F2/KIF18A/KIF4A/AURKA/MKI67/ASPM/KIF15/CENPF/STAG3/CENPE
## GO:1903047
CCNE1/LPIN1/E2F3/CDC20/CENPA/CDCA2/CDCA8/CHEK1/TTK/CDC25A/AURKB/STMN1/SKP
2/KIF2C/EGFR/FOXM1/NDC80/MELK/CCNB2/EZH2/CEP55/PLK1/STIL/CDK6/ANLN/CCNA2/
KIFC1/NFIB/BUB1/CDC45/TGFA/SKA3/MYBL2/SKA1/TRIP13/TPX2/CDCA5/BLM/CDC25B/B
IRC5/DLGAP5/CDKN2A/NCAPH/UBE2C/CDT1/KIF18B/NCAPG/KIF14/KIF20A/NUF2/KIF18A
/KIF4A/AURKA/MKI67/EIF4EBP1/KIF15/ID4/RRM2/CENPF/CENPE/IQGAP3

dim(ego)

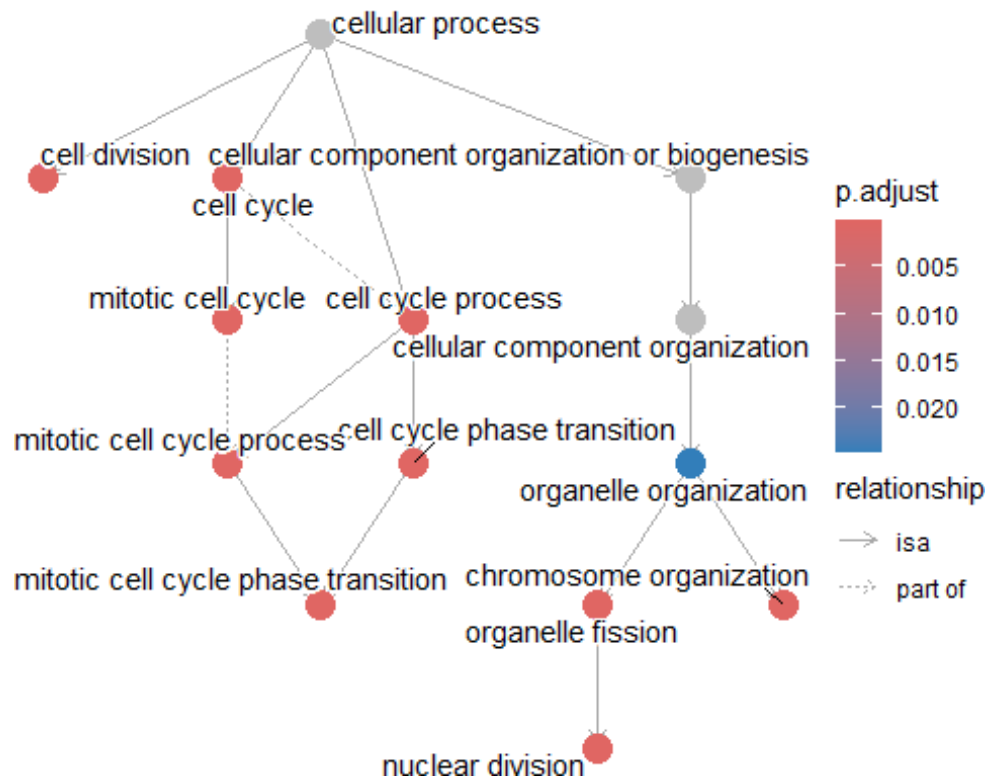
## [1] 35 11

```

Por lo tanto, mediante este análisis de enriquecimiento, vemos que la función que los genes significativos que hemos obtenido se dan en 3 procesos biológicos clave: la división nuclear (59 genes asociados), la organización del cromosoma (51 genes asociados) y el proceso del ciclo celular mitótico (82 genes asociados), todos con una significancia muy alta, lo que indica que dichos términos están muy enriquecidos por los genes asociados. Además, todos **forman una vía metabólica**, lo cual aporta consistencia biológica.

Hacemos ahora una representación visual de esta red de términos asociados.

```
goplot(ego)
```



Cuanto más rojo el término, más significativo es en el proceso biológico en cuestión. Vemos que los genes significativos para el receptor de estrógenos están altamente asociados con procesos celulares, desde la mitosis (reparto del material genético a

dos células hijas) hasta el mismo ciclo celular y la división celular. Además, estos genes están también involucrados en la división nuclear, la organización de los cromosomas y de los orgánulos, como se puede ver en la otra rama.

Ambas ramas tienen como término ancestro común los procesos celulares, pero no es significativo, al igual que la biogénesis.

Base de datos KEGG

Para ver las rutas metabólicas con mejor claridad, vamos a graficar estos términos utilizando la base de datos KEGG (Kyoto Encyclopedia of Genes and Genomes). En este caso, debemos convertir previamente los nombres de los genes en ENTREZ IDs.

```
# Añadir ENTREZ IDs a Los datos
id_conversion <- bitr(
  geneID = signif_genes_df$ID,
  fromType = "SYMBOL",
  toType = "ENTREZID",
  OrgDb = org.Hs.eg.db
)

## 'select()' returned 1:1 mapping between keys and columns

## Warning in bitr(geneID = signif_genes_df$ID, fromType = "SYMBOL",
## toType =
## "ENTREZID", : 12.47% of input gene IDs are fail to map...

# Fusionar con Los datos originales
genes_entrez <- merge(signif_genes_df, id_conversion,
                      by.x = "ID", by.y = "SYMBOL", all.x = FALSE)

# Comprobar en cuantos genes ha funcionado la conversion
cat("Exito de la conversion:",
    nrow(genes_entrez)/nrow(signif_genes_df)*100, "%\n")

## Exito de la conversion: 87.52588 %

# Lo convertimos en un vector
genes_entrez_vec <- sign(genes_entrez$logFC) * (-
log10(genes_entrez$P.Value))
# Le damos nombre a Los genes
names(genes_entrez_vec) <- rownames(genes_entrez)
# Ordenar en orden descendente
genes_entrez_vec <- sort(genes_entrez_vec, decreasing = TRUE)
# Comprobar el objeto
str(genes_entrez_vec)

## Named num [1:1691] 118 110 108 102 101 ...
## - attr(*, "names")= chr [1:1691] "1184" "130" "1593" "716" ...
```

Ahora que tenemos el ENTREZ ID de los datos, podemos usar la base de datos KEGG.

```

# GSEA - KEGG vías metabólicas
kegg1 <- gseKEGG(geneList = genes_entrez_vec,
                 organism = "hsa",
                 keyType = "kegg",
                 minGSSize = 10,
                 maxGSSize = 300,
                 pvalueCutoff = 0.05,
                 pAdjustMethod = "BH")

## Reading KEGG annotation online:
"https://rest.kegg.jp/link/hsa/pathway"...

## Reading KEGG annotation online:
"https://rest.kegg.jp/list/pathway/hsa"...

## using 'fgsea' for GSEA analysis, please cite Korotkevich et al (2019).
## preparing geneSet collections...
## GSEA analysis...
## no term enriched under specific pvalueCutoff...

head(kegg1, 3)

## [1] ID          Description      setSize      enrichmentScore
## [5] NES          pvalue         p.adjust     qvalue
## <0 rows> (or 0-length row.names)

```

No sale ninguna vía metabólica asociada a las funciones de los genes significativos, debe de ser debido a algún error técnico no detectado.

Análisis de sobrerrepresentación (ORA)

De manera complementaria, se comprobará la sobrerrepresentación de una lista de genes de una distribución en la que todos tienen anotación, para lo que se calcula el p-valor de que un término esté enriquecido, pero más adelante se tendrá que corregir.

En este caso, no tenemos que convertir el objeto en un vector, pero tenemos que definir el universo de genes y el método para ajustar el p-valor: usaremos `fdr`.

En este caso, necesitamos un vector con únicamente los nombres de los genes significativos, sin ningún orden concreto.

```

# Creación del vector de caracteres
signif_genes_names <- names(signif_genes)

# Enriquecimiento en GO, utilizando el método FDR
ego2 <- enrichGO(gene = signif_genes_names,
                 universe = rownames(de_res),

```

```

OrgDb = org.Hs.eg.db,
ont = "BP",
keyType = "SYMBOL",
pAdjustMethod = "BH", # Benjamin-Hochberg
pvalueCutoff = 0.001, # Reducir el numero de genes
asociados
qvalueCutoff = 0.01,
readable = TRUE)
head(ego2, 3)
##                               ID                               Description GeneRatio
BgRatio
## G0:0010817 G0:0010817 regulation of hormone levels 100/1539
370/11816
## G0:0008544 G0:0008544 epidermis development 72/1539
259/11816
## G0:0019730 G0:0019730 antimicrobial humoral response 30/1539
69/11816
##                               RichFactor FoldEnrichment      zScore      pvalue
p.adjust
## G0:0010817 0.2702703      2.075058 8.130340 2.074260e-13 1.052272e-
09
## G0:0008544 0.2779923      2.134345 7.142914 1.316706e-10 3.339824e-
07
## G0:0019730 0.4347826      3.338136 7.537613 3.970801e-10 6.714624e-
07
##                               qvalue
## G0:0010817 9.203163e-10
## G0:0008544 2.921007e-07
## G0:0019730 5.872605e-07
##
geneID
## G0:0010817
SOX11/SRD5A1/ECE2/SLC7A5/GAL/SLC16A10/RDH10/CPA4/LYN/EGFR/SOX8/TRPV6/KLK6
/SFRP1/NMB/BACE2/HSD17B2/SLC16A1/EFNA5/DGAT2/ADM/CD38/VSNL1/PCSK1N/IL4I1/
ALDH1A3/PLA2G7/SLC04A1/FOXD1/BMP2/DUOXA1/TNF/LRAT/CHST9/RIMS2/IYD/TFAP2B/
ADIPOQ/UGT2B11/ADH1B/DOC2B/ADRA2C/ABCA12/CYP26A1/NNAT/GDF9/WNT4/SDR16C5/C
RABP2/PDE8B/GCNT4/CTSG/LIPE/FFAR2/GJA1/GRP/DIO1/SLC30A8/GHR/PTPRN2/C1QTNF
3/CHST8/DHRS2/F2RL2/ADCY5/SLC18A2/SMPD3/CACNA1H/AGTR1/ADRA2A/UGT2B15/RAPG
EF3/PCSK6/HPN/GIPR/HSD17B8/CYP21A2/BAIAP3/STC2/TBX3/CPA3/ITPR1/IRS1/WNK4/
RPH3AL/MYRIP/IGF1R/PNPLA4/CPLX1/PCSK4/SYBU/ABCC8/SYTL4/SLC7A8/MYB/ABAT/KC
NJ11/FOXA1/GATA3/ESR1
## G0:0008544
YBX1/FOXC1/PLOD1/KRT16/GAL/CDH3/STMN1/PPARA/EGFR/EZH2/FERMT1/KRT81/TGM1/K
RT6C/MET/WNT10A/KRT86/FABP5/KRT6A/PKP1/KRT6B/SMO/GATA6/KRT7/DKK1/KLK7/PTC
H1/CALML5/KLK5/SOSTDC1/SOX9/CLDN1/BCL11B/ETV4/ZNF750/KRT17/MYCN/ZBED2/GJB
5/TNF/KRT5/LAMC2/CSTA/FOXQ1/S100A7/GRHL3/LGR5/KRT15/HOXB13/KRT14/COL17A1/
ABCA12/TGM3/ALOXE3/CRABP2/IGFBP5/SLITRK6/LAMA3/DNASE1L2/SVEP1/BMP4/SULT2B
1/EDA2R/ANKRD24/ASAH1/PLA2G10/TPRN/IL20/UGCG/MSX2/BCL2/SLC44A4
## G0:0019730

```

```
SLPI/CXCL1/PI3/MMP7/S100A9/DEFB1/CX3CL1/KRT6A/CCL18/GATA6/ADM/CXCL10/KLK7
/KLK5/CCL13/CCL8/CXCL11/GNLY/S100A7/CXCL13/CXCL2/TF/CXCL9/CCL17/WFDC2/CCL
14/CTSG/CXCL14/CST9/RNASE4
##          Count
## G0:0010817    100
## G0:0008544     72
## G0:0019730     30

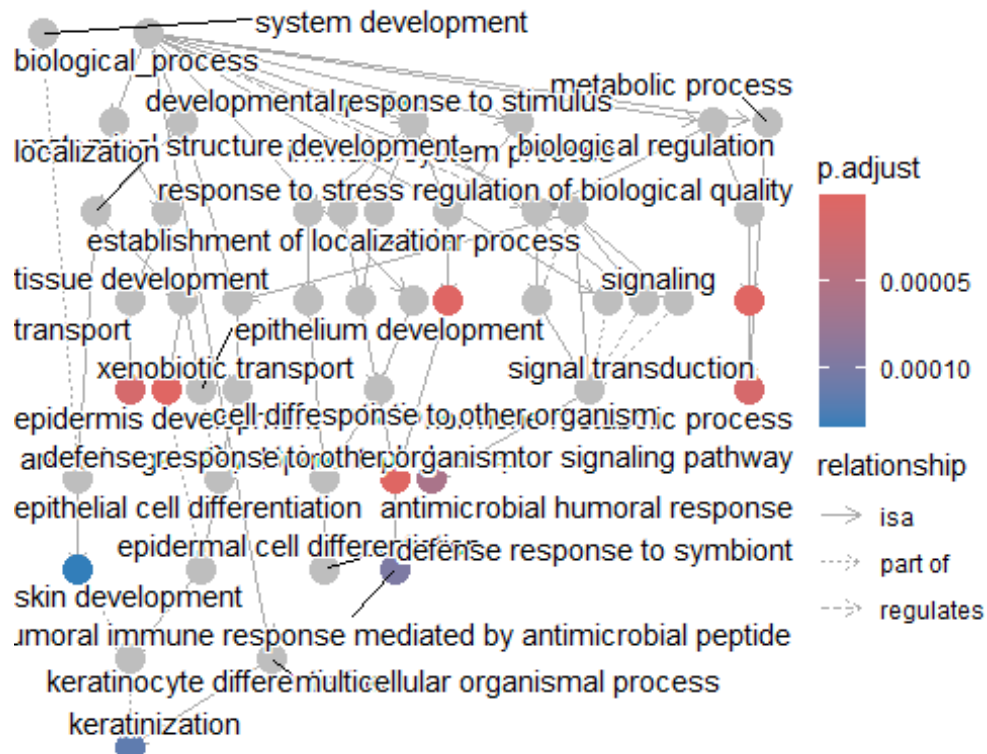
dim(ego2)
## [1] 34 12
```

En este caso, vemos que los procesos biológicos más importantes en los que tienen función los genes significativos para ER en cáncer de mama son estos 3: regulación de los niveles hormonales, desarrollo de la epidermis y la respuesta humoral antimicrobiana.

La columna GeneRatio muestra la cantidad de genes que están expresados en la proteína que actúa en dicha función dentro de los genes significativos, mientras que la columna BgRatio muestra lo mismo pero en todo el universo. Son estos dos ratios los que se van a comparar para ver si la función está sobrerrepresentada o no, lo cual se mira con el p-valor ajustado. Las tres funciones tienen un p-valor significativo, lo que nos indica que están muy sobrerrepresentadas en el set de genes significativos, donde el primer proceso biológico, **regulación de niveles de hormonas**, es el que está más sobrerrepresentado, lo cual tiene una alta implicación biológica.

Vamos a verlo visualmente.

```
gplot(ego2, max.overlaps = Inf)
## Warning: ggrepel: 13 unlabeled data points (too many overlaps).
Consider
## increasing max.overlaps
```



En este caso, aparecen muchos términos en color gris, lo que significa que hay una gran cantidad de genes de la lista que no están anotados en ninguno de esos términos GO significante, de manera que las vías metabólicas no están enriquecidas con el set de genes con el que trabajamos. Esto es normal y esperado, ya que este análisis de sobrerrepresentación sólo destaca genes que conducen a un enriquecimiento significativo, de manera que dichos genes serán “silenciosos” para esos procesos concretos.

Base de datos KEGG

Vamos a comprobar qué ocurre con la base de datos KEGG.

```
# ORA - KEGG vías metabólicas
kegg2 <- enrichKEGG(gene = genes_entrez$ENTREZID,
  organism = "hsa",
  minGSSize = 50,
  maxGSSize = 300,
  pvalueCutoff = 0.05,
  pAdjustMethod = "BH")

head(kegg2)

##                                category
## hsa04382                          <NA>
## hsa03320          Organismal Systems
## hsa04061 Environmental Information Processing
```

```

## hsa04934 Human Diseases
## hsa04814 Cellular Processes
## hsa04915 Organismal Systems
## subcategory ID
## hsa04382 <NA> hsa04382
## hsa03320 Endocrine system hsa03320
## hsa04061 Signaling molecules and interaction hsa04061
## hsa04934 Endocrine and metabolic disease hsa04934
## hsa04814 Cell motility hsa04814
## hsa04915 Endocrine system hsa04915
## Description
## hsa04382 Cornified envelope formation
## hsa03320 PPAR signaling pathway
## hsa04061 Viral protein interaction with cytokine and cytokine receptor
## hsa04934 Cushing syndrome
## hsa04814 Motor proteins
## hsa04915 Estrogen signaling pathway
## GeneRatio BgRatio RichFactor FoldEnrichment zScore
pvalue
## hsa04382 40/824 217/9561 0.1843318 2.138830 5.211154
2.876747e-06
## hsa03320 19/824 76/9561 0.2500000 2.900789 5.108970
1.636877e-05
## hsa04061 22/824 100/9561 0.2200000 2.552694 4.793236
3.190210e-05
## hsa04934 29/824 155/9561 0.1870968 2.170913 4.513353
4.965719e-05
## hsa04814 34/824 197/9561 0.1725888 2.002575 4.366472
6.475344e-05
## hsa04915 26/824 139/9561 0.1870504 2.170374 4.268468
1.206182e-04
## p.adjust qvalue
## hsa04382 0.0006760356 0.0005147863
## hsa03320 0.0019233305 0.0014645742
## hsa04061 0.0024989977 0.0019029322
## hsa04934 0.0029173599 0.0022215059
## hsa04814 0.0030434116 0.0023174915
## hsa04915 0.0047242142 0.0035973859
##
geneID
## hsa04382
26154/59344/9076/137075/9073/9080/1308/1286/1287/1288/126410/1828/1830/38
61/3866/3868/3872/3875/25984/3852/3853/3854/286887/3855/3887/3892/3909/39
18/11240/5046/5317/6271/6281/6273/6278/6279/6280/7051/7053/7357
## hsa03320
8309/23305/9370/51129/364/2167/2171/2173/9415/3158/4199/4312/5105/5346/12
3/729359/440503/5465/11001
## hsa04061
6357/6358/6361/6362/6355/6376/1524/2919/3627/6373/10563/9547/2920/4283/50
604/53832/53833/3559/146433/3572/4049/7124

```

```
## hsa04934
107/111/115/185/8313/776/778/8913/8912/816/595/898/1021/1029/90993/148327
/1589/1870/1871/1956/3708/5087/50940/8622/5332/83439/80326/54361/7475
## hsa04814
72/1062/8701/1767/1768/56171/27019/83544/7802/79659/83657/113220/23303/99
28/56992/55614/81930/146909/547/10112/11004/24137/3800/3833/4629/4651/140
469/55930/7137/7138/113457/112714/7277/10381
## hsa04915
107/111/115/596/810/51806/90993/148327/1956/2099/9687/3708/3760/3861/3866
/3868/3872/3875/25984/5241/5332/5914/25759/399694/7031/7039
##          Count
## hsa04382      40
## hsa03320      19
## hsa04061      22
## hsa04934      29
## hsa04814      34
## hsa04915      26

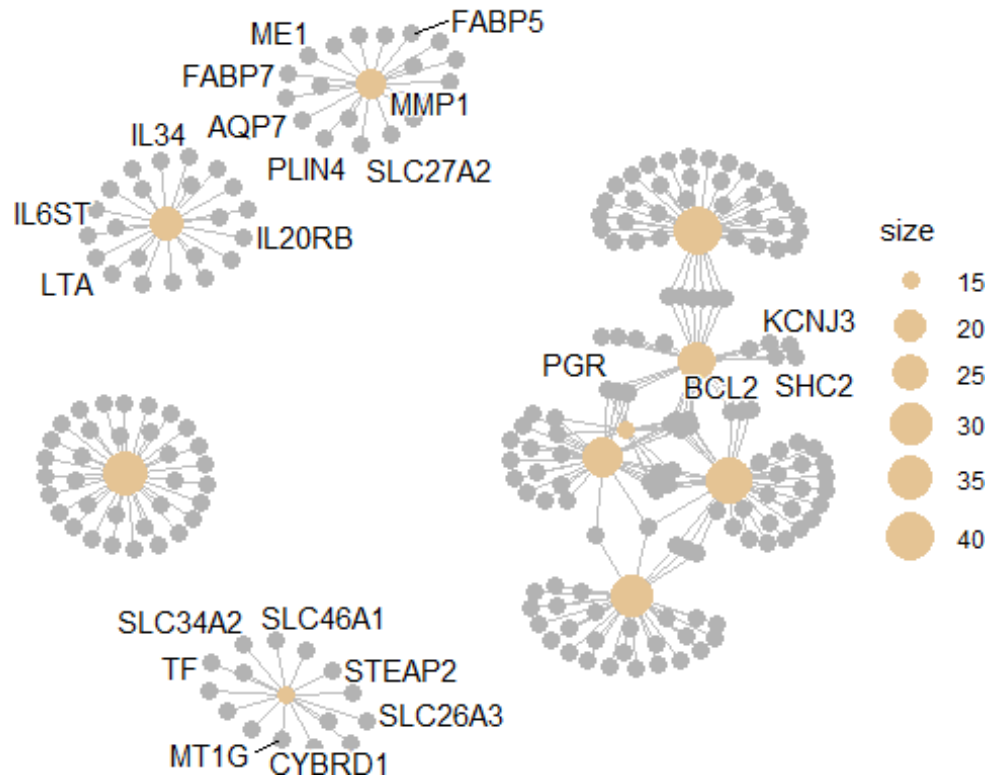
dev.new()
dotplot(kegg2, showCategory=12) + ggtitle("KEGG Pathways ORA")
```

Hay algunos términos que están estrictamente relacionados con las vías metabólicas sobre-enriquecidas por tumores de ER+, como son las **señales de transducción de estrógenos**, el cual es la vía metabólica que define tumores de mama con ER+; y las **señales de transducción de PPAR**, el cual es un marcador de cancer de mama luminal con ER+, pues PPAR está altamente expresado en tumores ER+. Otros términos están relacionados de manera indirecta por su relación hormonal, como la **síntesis y secreción de la aldosterona** o la **síntesis y secreción del cortisol**, los cuales son vías metabólicas de la hormona esteroide (y glucocorticoide en el segundo caso) que comparte mecanismos regulatorios con los estrógenos; y la señalización de la oxitocina, que es una vía metabólica de la hormona reproductiva expresada en el tejido mamario.

Para obtener más información de estos términos, vamos a realizar un cnetplot de esto.

```
kegg2R <- setReadable(kegg2, "org.Hs.eg.db", "ENTREZID")
cnetplot(kegg2R, showCategory = 10)

## Warning: ggrepel: 209 unlabeled data points (too many overlaps).
## Consider
## increasing max.overlaps
```



Esto nos muestra 4 módulos funcionales principales que diferencian ER+ y ER- en el cáncer de mama:

- **Cluster 1 (a la derecha, el más grande): señalización de estrógenos y respuesta hormonal.** Se ven genes importantes como ESR1 o TFF1. Es el núcleo de la firma de los tumores con ER+.
- **Cluster 2 (arriba en el medio): metabolismo de lípidos y señalización de PPAR.** Acumula genes importantes para estos mecanismos como PPAR o ácidos grasos (FABP5, FABP7). Representa el fenotipo lipogénico y metabólico del cancer de mama luminal con ER+.
- **Cluster 3 (arriba a la izquierda): señalización inmune e inflamatoria.** Aparecen citoquinas y quimioquinas esenciales como la familia de las CCL o las CXCL. Estas representan diferencias en los microentornos inmunes entre tumores de ER+ y ER-.
- **Cluster 4 (abajo a la derecha): transporte de iones y homeostasis.** Transportadores como la familia de las SLC e iones ATPasas de Na⁺ y K⁺, entre otras. Representan funciones que apoyan al metabolismo, como el transporte de nutrientes o la homeostasis iónica.

Este gráfico es muy importante, mostrando una caracterización a múltiples niveles moleculares que demuestra que el estatus de ERno sólo define la respuesta

hormonal, sino también el metabolismo fundamental y las propiedades de los tumores inmunológicos.

Conclusiones

Se obtienen interesantes conclusiones de este análisis de expresión diferencial y de enriquecimiento funcional. Para empezar, obtuvimos que el gen ESR1 es el que más diferencialmente expresado está, además de ser el más significativo, como se puede ver en el volcanoplot, lo cual es un hallazgo muy importante, pues es el gen que se expresa en la proteína encargada de la regulación de estrógenos. Sin embargo, se ven algunos resultados un tanto inquietantes, como la similitud de 1 que tienen genes como TFF1 o TFF3 con la mayoría, lo cual, al ser casos tan puntuales, puede estar indicando algún error técnico en los valores de expresión de dichos genes.

Por otro lado, en el estudio de los procesos biológicos a los que están asociados estos genes, se obtienen resultados con altas implicaciones biológicas con la base de datos GO. Tristemente, no se pudieron comparar los resultados con la base de datos KEGG por problemas técnicos. El análisis de enriquecimiento muestra procesos biológicos con una alta significancia que, además, forman una vía metabólica altamente asociada a los procesos de división celular, nuclear y transmisión de la información genética, lo que tiene una gran importancia en la proliferación del cáncer. En particular, las células cancerígenas con **ER+** de cáncer de mama son un subtipo clínico que muestra una **menor proliferación** que los tumores ER-, por lo que la división celular y la mitosis son procesos altamente enriquecidos en muestras con ER-.

Por último, los resultados de ORA muestran un **patrón biológicamente coherente** para cáncer de mama con ER+, especialmente por la **regulación de hormonas**, ya que los tumores de ER+ mantienen la respuesta hormonal luminal, lo que implica que si se pierde la regulación de la pérdida de hormonas, habrá una transición a un estado de tipo basal. Los hallazgos de términos como *desarrollo de la epidermis* o *respuesta humoral antimicrobiana* son un poco más inesperados, aunque pueden estar relacionados, en el primer caso, por la supresión de la diferenciación basal/epidermal en tumores ER+, lo cual es un marcador de la identidad luminal; y en el segundo, debido al microentorno inmune al frío de los tumores ER+ frente a los tumores ER-, que son inmunes al calor.

En general, los resultados obtenidos son biológicamente coherentes y con un potencial clínico muy grande, pudiendo describir un **fenotipo del tumor ER+ de cáncer de mama:**

Las células luminales epiteliales mantienen la respuesta hormonal, se suprimen los programas de diferenciación basal y residen en un microentorno inmune al frío.

Esto se alinea perfectamente con los subtipos Luminal A/B de PAM50, que además queda bien interpretado mediante el cnet plot, donde la relevancia clínica y biológica adquieren gran valor.

`sessionInfo()`

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
## LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_United Kingdom.utf8
## [2] LC_CTYPE=English_United Kingdom.utf8
## [3] LC_MONETARY=English_United Kingdom.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United Kingdom.utf8
##
## time zone: Europe/Madrid
## tzcode source: internal
##
## attached base packages:
## [1] stats4      stats      graphics  grDevices  utils      datasets
##      methods
## [8] base
##
## other attached packages:
##  [1] stringr_1.6.0      tidyr_1.3.2      readr_2.1.6
##  [4] tinytex_0.58       knitr_1.51       dplyr_1.2.0
##  [7] clusterProfiler_4.16.0 DOSE_4.2.0      org.Hs.eg.db_3.21.0
## [10] AnnotationDbi_1.70.0 IRanges_2.42.0  S4Vectors_0.46.0
## [13] Biobase_2.68.0      BiocGenerics_0.56.0 generics_0.1.4
## [16] GOSemSim_2.34.0     pheatmap_1.0.13 ggrepel_0.9.6
## [19] ggplot2_4.0.2       edgeR_4.6.3     limma_3.64.3
##
## loaded via a namespace (and not attached):
##  [1] DBI_1.2.3          gson_0.1.0       rlang_1.1.7
##  [4] magrittr_2.0.4     otel_0.2.0       compiler_4.5.2
##  [7] RSQLite_2.4.5      png_0.1-8        vctrs_0.7.1
## [10] reshape2_1.4.5     pkgconfig_2.0.3  crayon_1.5.3
## [13] fastmap_1.2.0      XVector_0.48.0   labeling_0.4.3
## [16] rmarkdown_2.30     tzdb_0.5.0       enrichplot_1.28.4
## [19] UCSC.utils_1.4.0   purrr_1.2.1      bit_4.6.0
## [22] xfun_0.56          cachem_1.1.0     aplot_0.2.9
## [25] GenomeInfoDb_1.44.3 jsonlite_2.0.0   blob_1.3.0
## [28] BiocParallel_1.42.1 parallel_4.5.2   R6_2.6.1
## [31] stringi_1.8.7      RColorBrewer_1.1-3 Rcpp_1.1.1
## [34] snow_0.4-4         ggtangle_0.1.1   R.utils_2.13.0
```

## [37] Matrix_1.7-4	splines_4.5.2	igraph_2.2.1
## [40] tidyselect_1.2.1	qvalue_2.40.0	rstudioapi_0.18.0
## [43] yaml_2.3.12	codetools_0.2-20	lattice_0.22-7
## [46] tibble_3.3.1	plyr_1.8.9	treeio_1.32.0
## [49] withr_3.0.2	KEGGREST_1.48.1	S7_0.2.1
## [52] evaluate_1.0.5	gridGraphics_0.5-1	Biostrings_2.76.0
## [55] pillar_1.11.1	ggtree_3.16.3	ggfun_0.2.0
## [58] vroom_1.7.0	hms_1.1.4	scales_1.4.0
## [61] tidytree_0.4.7	glue_1.8.0	ggarchery_0.4.4
## [64] lazyeval_0.2.2	tools_4.5.2	ggnewscale_0.5.2
## [67] data.table_1.18.2.1	fgsea_1.34.2	locfit_1.5-9.12
## [70] fs_1.6.6	fastmatch_1.1-8	cowplot_1.2.0
## [73] grid_4.5.2	ape_5.8-1	colorspace_2.1-2
## [76] nlme_3.1-168	GenomeInfoDbData_1.2.14	patchwork_1.3.2
## [79] cli_3.6.5	rappdirs_0.3.4	gtable_0.3.6
## [82] R.methodsS3_1.8.2	yulab.utils_0.2.4	digest_0.6.39
## [85] ggplotify_0.1.3	farver_2.1.2	memoise_2.0.1
## [88] htmltools_0.5.9	R.oo_1.27.1	lifecycle_1.0.5
## [91] http_1.4.7	GO.db_3.22.0	statmod_1.5.1
## [94] bit64_4.6.0-1		

Bibliografía

1. Creighton et al., *Breast Cancer Res* (2009): ER+ tumors show luminal differentiation signatures; ER- show proliferation signatures
2. The Cancer Genome Atlas Network, *Nature* (2012): Basal-like tumors enriched in cell cycle pathways; Luminal tumors enriched in hormone response
3. Sørlie et al., *PNAS* (2001): Original molecular subtypes – Luminal A (low proliferation) vs. Basal-like (high proliferation)
4. Perou CM, Sørlie T, Eisen MB, et al. (2000). Molecular portraits of human breast tumours. *Nature*
5. Heldring N, Pike A, Andersson S, et al. (2007). Estrogen receptors: how do they signal and what are their targets. *Physiol Rev*
6. <https://yulab-smu.top/biomedical-knowledge-mining-book/021-go.html#go-gene-set-enrichment-analysis>
7. <https://www.ub.edu/nurcamein/es/receptores-nucleares/er-receptor-de-estrogenos/>

Declaración de uso de IA

- Comprensión y correcto procedimiento para la descarga de los datos de GDC.

- Comprensión y corrección de errores de código a lo largo de todo el script.
- Resolución del problema de la columna 1 con el nombre de una muestra y la última columna con dos valores. Esto me tomó más tiempo de lo que pensaba.
- Comprensión de conceptos estadísticos y biológicos para el correcto desarrollo del análisis.
- Ayuda para a interpretación del cnet plot y para la discusión final.